neo4j Graph Intelligence Platform

CrabRAG

Why your assistant needs graph memory, not more tokens.

Stephen Chin
VP of Developer Relations, Neo4j · @steveonjava

This is Crabd.

He's a personal AI assistant

Autonomous, eager, plugged into all your tools.
Like every assistant on stage this week, he demos beautifully.

There is just one problem

Every session, he wakes up an amnesiac.

The only reason Clawd knows his own name is the file on the sink: `SOUL.md`

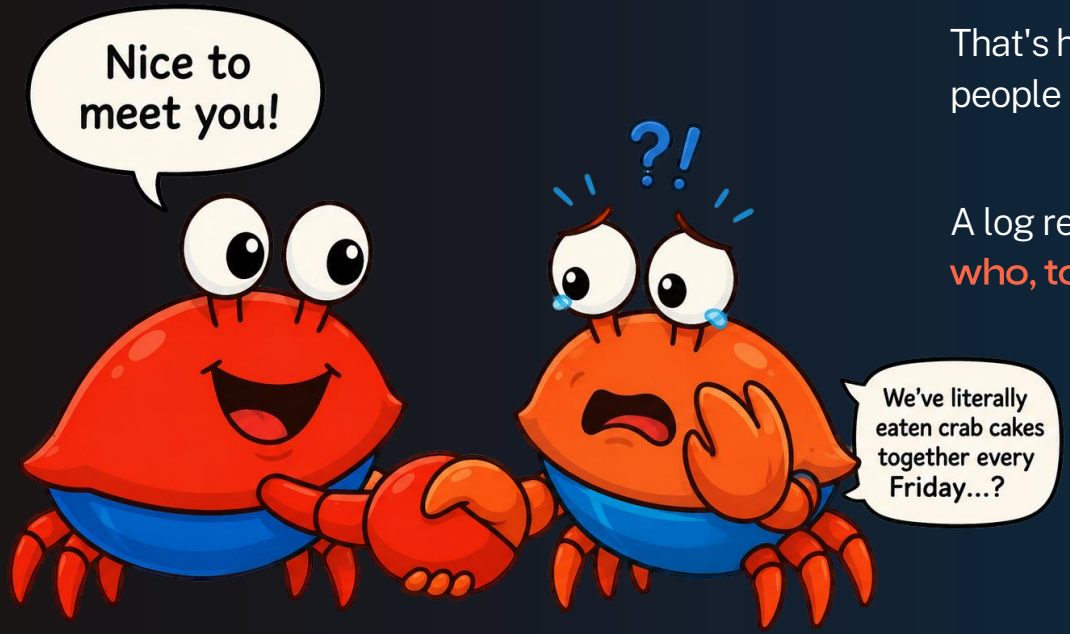
No hidden state. An OpenClaw agent only remembers what's saved to disk — and re-reads it from zero each time.

A drawer full of tools. Still can't eat soup.

Screwdriver? Wrench? Clawd has every tool an agent could want — and reaches for the wrong one anyway

The problem was never tool access. It's knowing which tool this moment calls for.

“Nice to meet you!” (for the 100th time)



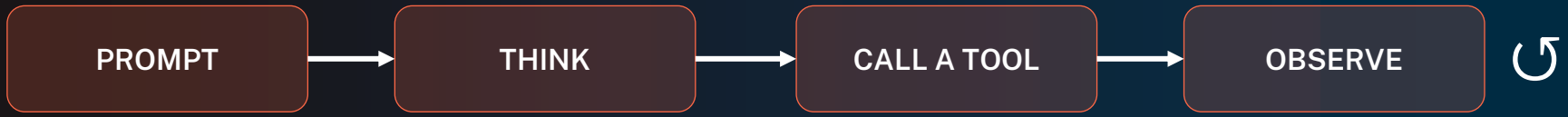
That's his best friend. Clawd has no memory of people — or how he's connected to them.

A log remembers that they talked. It forgets who, to whom, and why — the relationships.

So how does Crabd's brain actually work?

Spoiler: it's a folder full of Markdown files — and it re-reads all of them every morning.

The loop is easy.
The memory is the hard part.



MEMORY? What survives between turns? Between sessions? This is the part everyone hand-waves — and the part that makes assistants flaky.

The files are the agent.

`SOUL.md`

Identity, personality, boundaries. Injected every session.

`AGENTS.md`

Identity, personality, boundaries. Injected every session.

`USER.md`

Identity, personality, boundaries. Injected every session.

`MEMORY.md`

Identity, personality, boundaries. Injected every session.

`TOOLS.md`

Identity, personality, boundaries. Injected every session.

`memory/2026-06-22`

Identity, personality, boundaries. Injected every session.

Open them in any editor and you're editing Clawd's mind.
Transparent, Git-able — and entirely flat.

5m

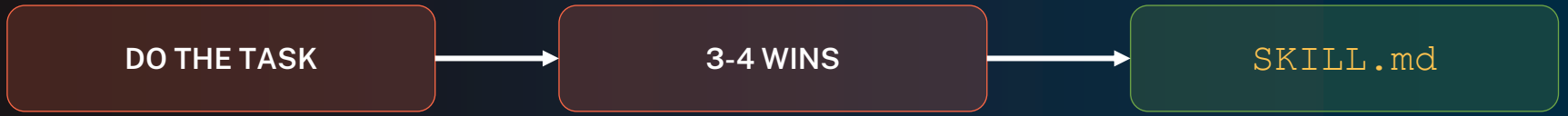
tokens. every day.

No structured memory means Clawd re-reads the entire stack every session. OpenClaw is anti-RAG by design — and the token bill shows it.

Don’t read more. Remember better.

Hermes writes its own skills.

Same `SOUL.md` identity. A curated, capped `MEMORY.md`.
Plus a learning loop : do the work, notice it worked, save the recipe.



↺ reused automatically next time

It remembers methods, not just facts — procedural memory.

PART TWO

Skills

Clawd learns to do things. What could go wrong?

Skills = folders with a `SKILL.md`. Bundled, installed, or self-written.
Three ways they break ↓

Great at shrimp. Defeated by a clam.

His shrimp skill is flawless. The clam is new.

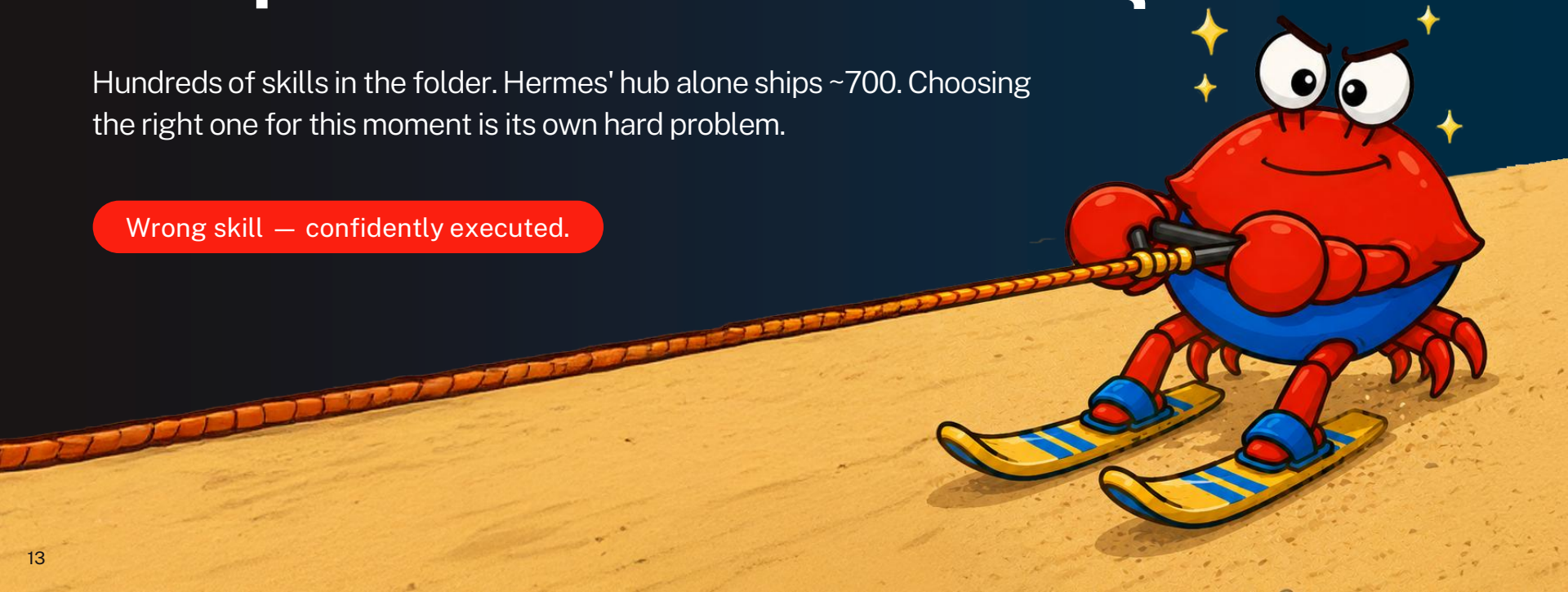
There is no `clam.md`, so he just . . . stops.

Skills aren't adaptive; one recipe, no fallback.

He meant to jog.
He picked waterskiing.

Hundreds of skills in the folder. Hermes' hub alone ships ~700. Choosing the right one for this moment is its own hard problem.

Wrong skill — confidently executed.



Cracks the shell. Wanders off.

He has `crack_shell` and `eat_meat`.
He just can't connect them.

Skills don't compose . Real work is a chain of steps — and the meat is inside the shell he just cracked.
(Hold that thought)

A THIRD ASSISTANT

Great at shrimp. Defeated by a clam.

Goose

An open-source AI agent that runs on your machine — desktop app, CLI, and API. Built in Rust, works with any LLM.

An Agentic AI Foundation (AAIF) project · Linux Foundation

Everything is an MCP extension.

70+ MCP extensions — browsers, databases, docs, tools

100% local — your machine, your keys, your files.

Memory is just another MCP server

Memory is just another MCP server.

`remember_memory()` `retrieve_memories()`

`remove_specific_memory()` `remove_memory_category()`

Driven by trigger words — say `remember…` and it writes a tagged entry; say `forget…` and it drops one. Stored as plain files, scoped local or global , sorted into categories and #tags.

Memories are plain files on disk

```
~/.config/goose/memory/ ├─ global/
preferences.md └─ local/ project-x.md
```

```
# development #rust #tooling - prefers
ripgrep over grep
```

local · this project

global · everywhere

The same catch.

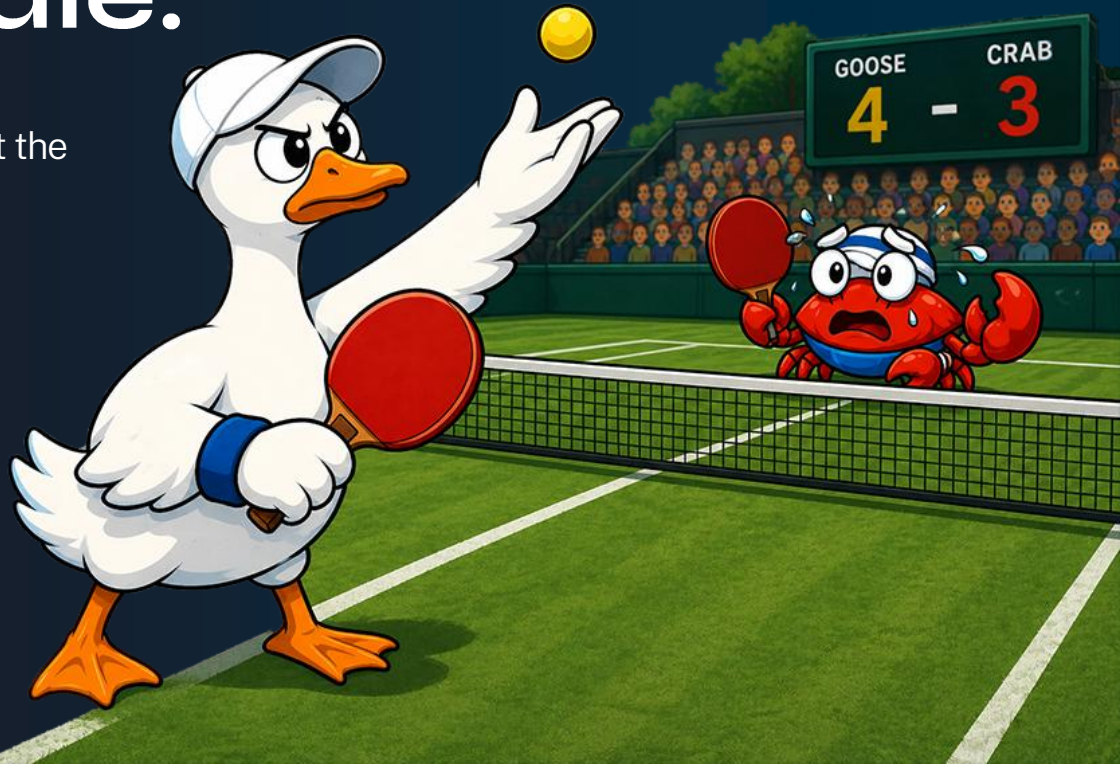
Goose loads **all** saved memories into **every** prompt, **every** session. More memory means more tokens on every call — and still no relationships.

A THIRD ASSISTANT

Right game. Wrong paddle.

MCP hands the agent a tool. It can't hand it the judgment to pick the right one.

...or maybe he just invented pickleball.



Remembers everything. Can’t take off.

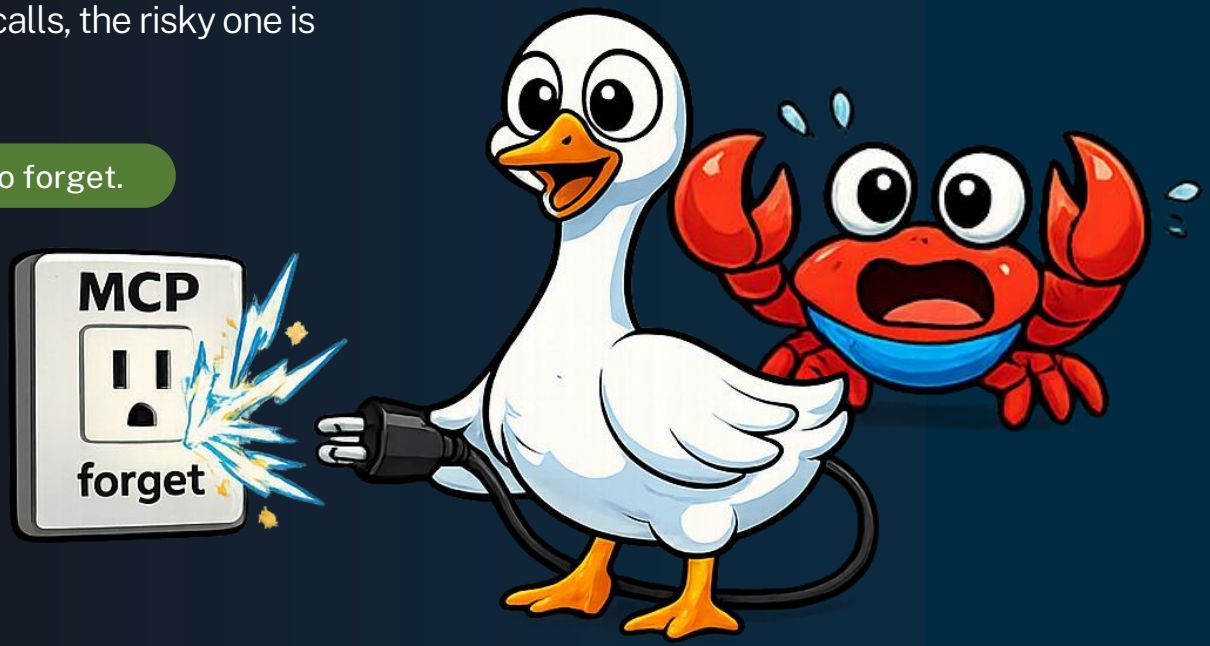
Every memory, every prompt, every time. Load the whole pile on each call and eventually the bird can’t fly.

Flat memory grows — it never gets lighter.

Plugging straight into the socket.

When memory is just loose tool calls, the risky one is always one call away.

Fortunately, tool calls are easy to forget.



“Just give him a vector database.”

Pinecone
Weaviate
Chroma
Qdrant
Milvus
pgvector
redis
Lance DB
FAISS
Mem0
Vespa
Marqo
+ a dozen more

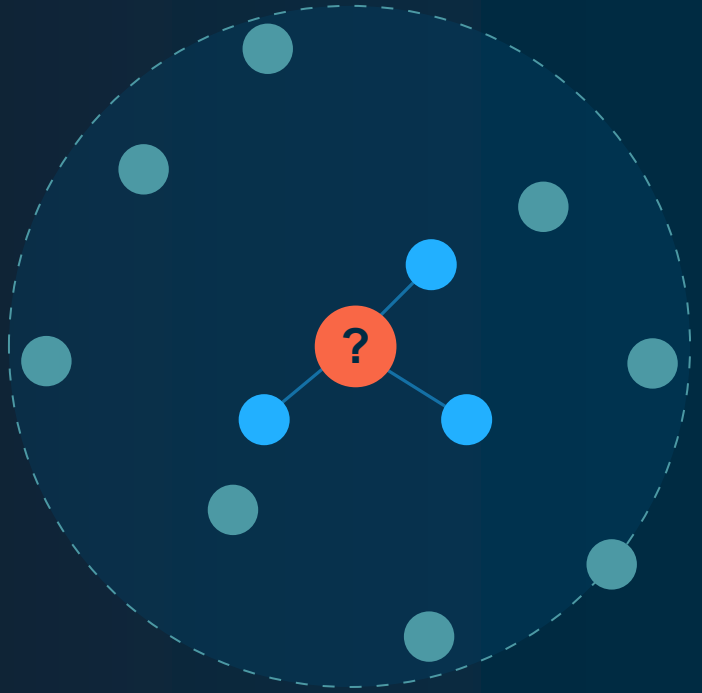
Pick one. Good Luck.

HOW SIMILARITY SEARCH WORKS

Find the nearest points in space.

Every note becomes a vector. Your query becomes a vector. Return the closest ones.

Excellent for **“what looks similar?”**



query → 3 nearest neighbors

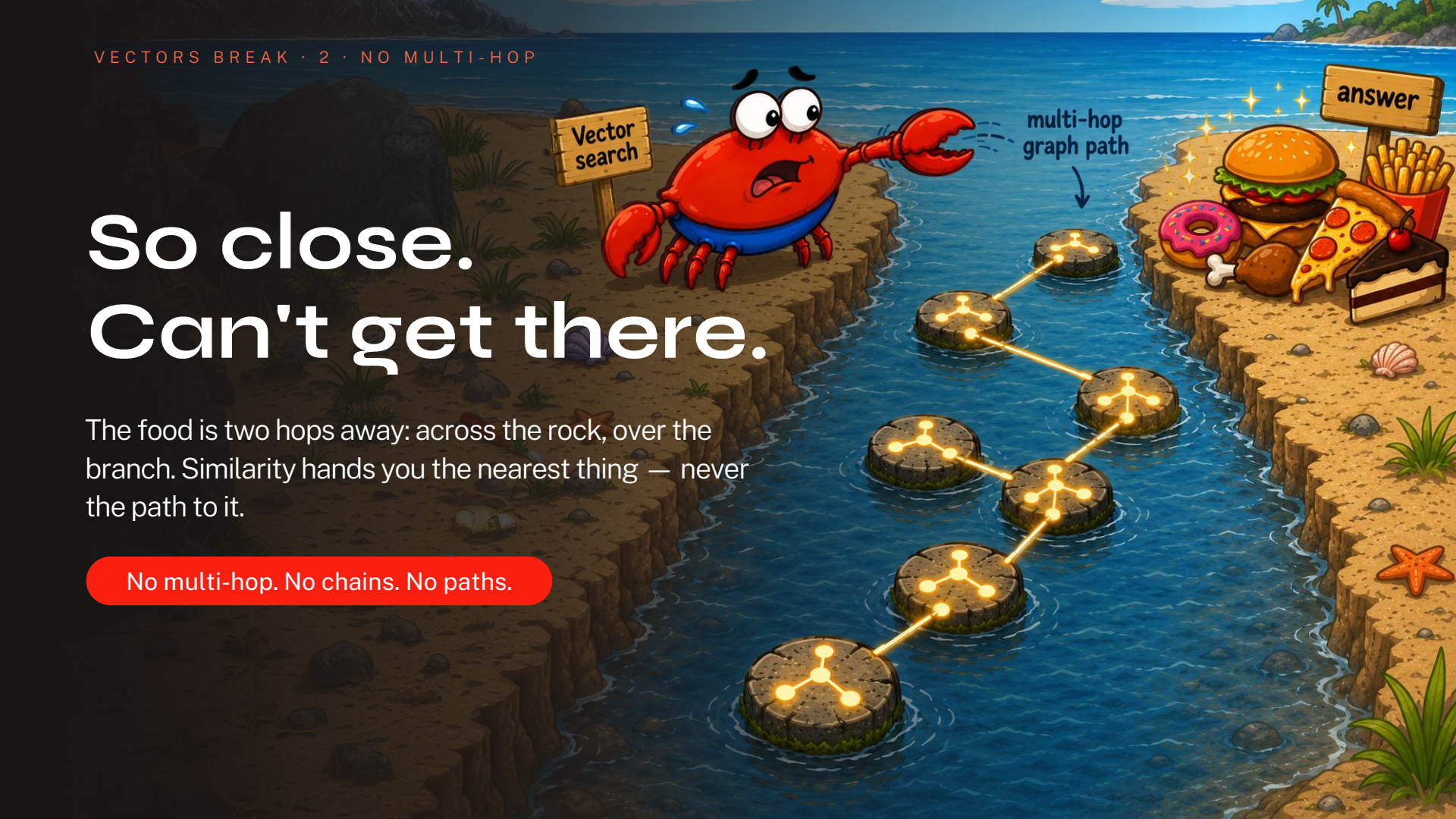
Similar ≠ related.
Nearest ≠ right.

Embeddings know when two things look alike.
They don't know that one caused, belongs to,
or connects to the other

He wanted an apple.

Similarity search returned the closest match: an Apple.
Different apple. He's three bites in.

Looks identical. Not remotely related.



So close. Can't get there.

The food is two hops away: across the rock, over the branch. Similarity hands you the nearest thing — never the path to it.

No multi-hop. No chains. No paths.

“Which shell is mine?”

Ten shells, all a 99% match. But “mine” isn't about looks — it's `(Clawd)-[:OWNS]→(shell)`.

Identity, ownership, history — all **relationships**. Similarity can't see them

PART FOUR

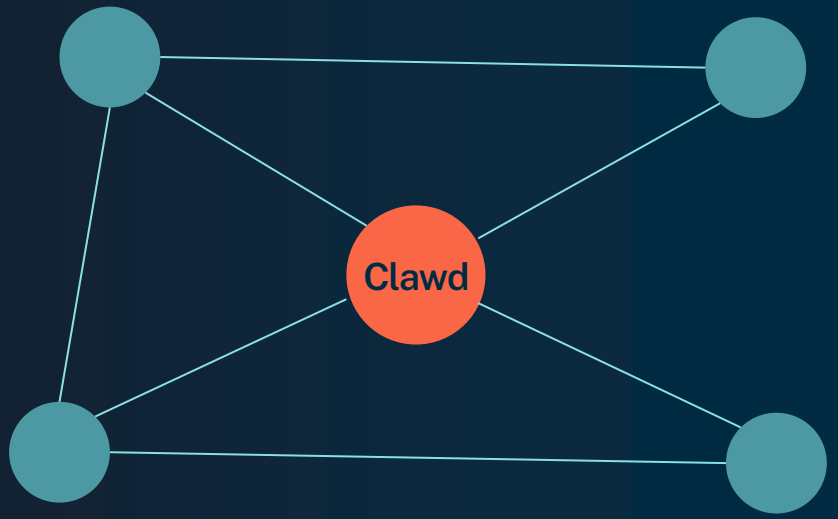
Enter the graph.

Relationships. Paths. Identity. The chain from crack to eat. A graph stores all of it — natively.

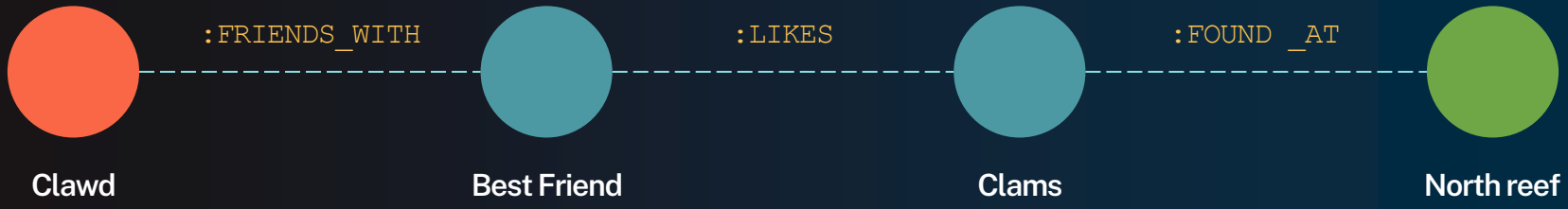
This is GraphRAG

Built for connected data.

Entities are nodes. Relationships are **first-class edges** — stored, typed, traversable. The part vectors discard is the part a graph is made of



Multi-hop answers in a single traversal.



“Where should I take my friend to eat? — one hop chain, one answer.

The answer is the explanation.

Accurate
Grounded in stored relationships, not a fuzzy nearest guess.

Explainable
The traversal path is the citation. Show your work, every time.

Auditable
When it's wrong, trace exactly which edge led astray.

CrabRAG · graph memory, not more tokens

LIVE DEMO · HOMELAB

There is no spoon.
There is only the graph.

01

Clawd writes each action into the graph as he works.

02

A follow-up is answered by traversal, not re-reading

03

Fresh session — he still remembers. The window reset; the graph didn't.

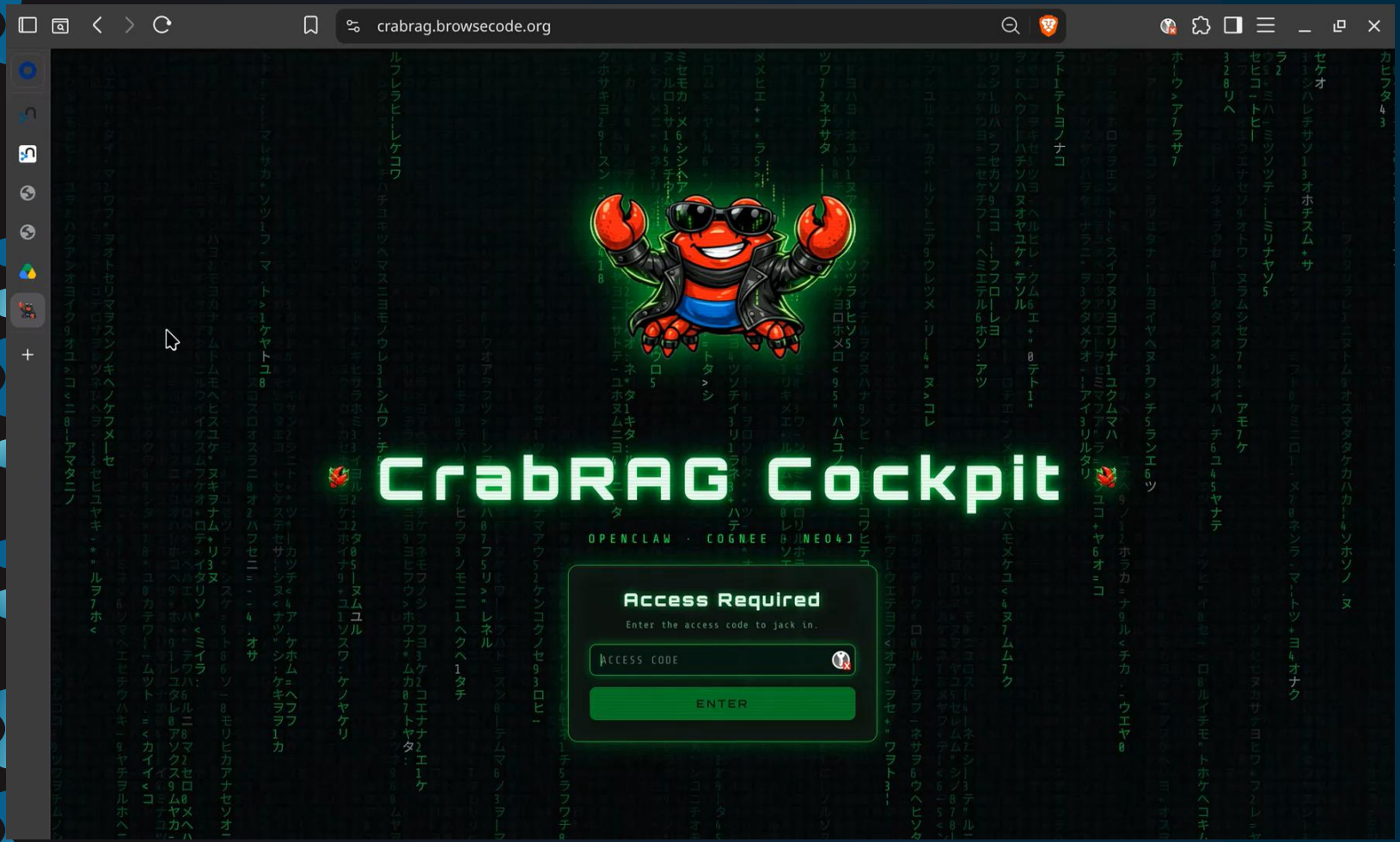
Homelab infrastructure

browsecode cluster + standalone proxmox · 4 hosts · 3-node cluster (PVE 9.2.3, quorate 3/2) · isolated graph-memory demo (VLAN100)

Homelab infrastructure

browsecode cluster + standalone proxmox · 4 hosts · 3-node cluster (PVE 9.2.3, quorate 3/2) · isolated graph-memory demo (VLAN100)

LIVE DEMO · HOMELAB

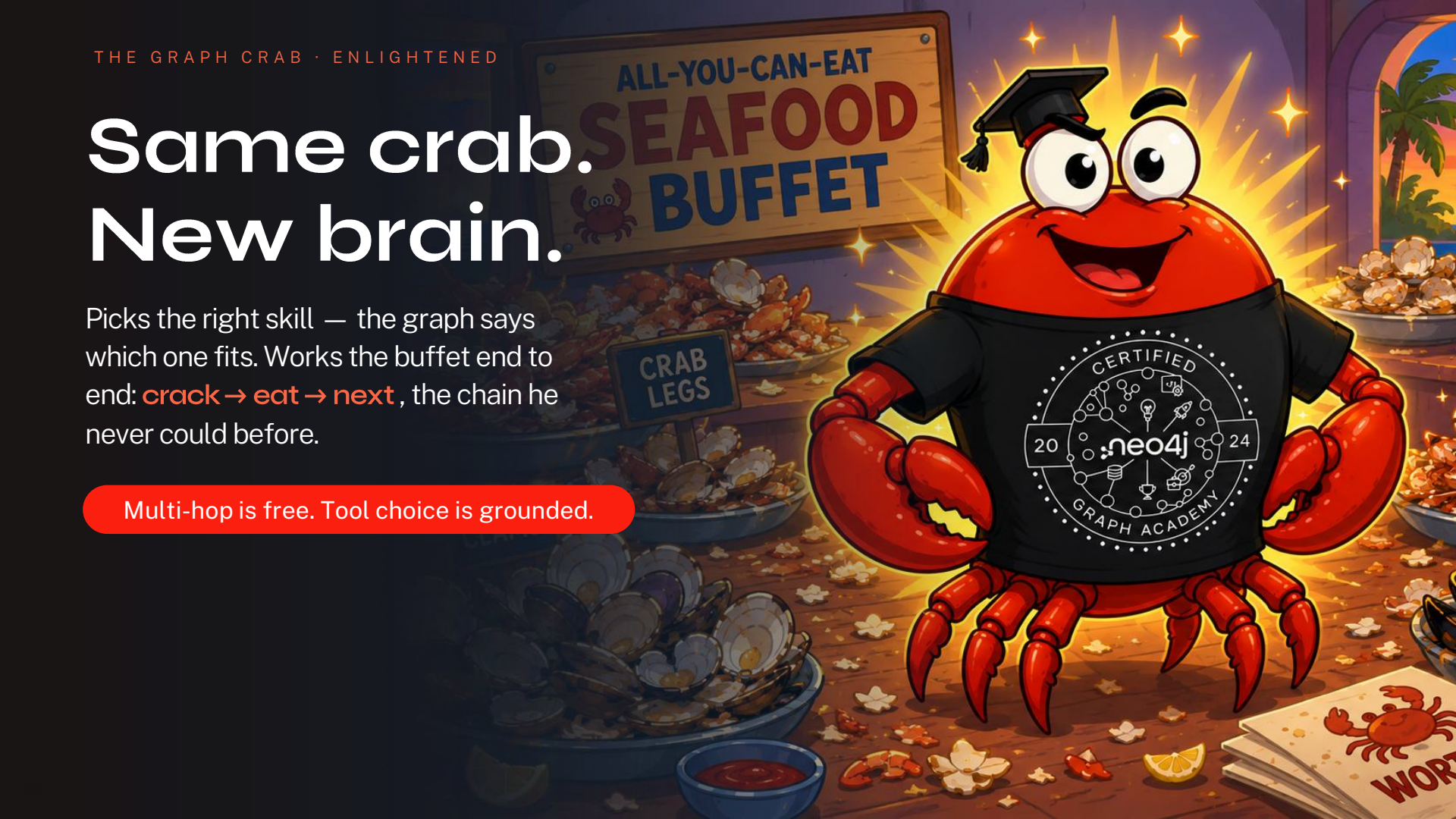

crabrag.browsecode.org
CrabRAG Cockpit
OPENCLAW · COGNEE · NEO4J
Access Required
Enter the access code to jack in.
ACCESS CODE
ENTER

THE GRAPH CRAB · ENLIGHTENED

Same crab. New brain.

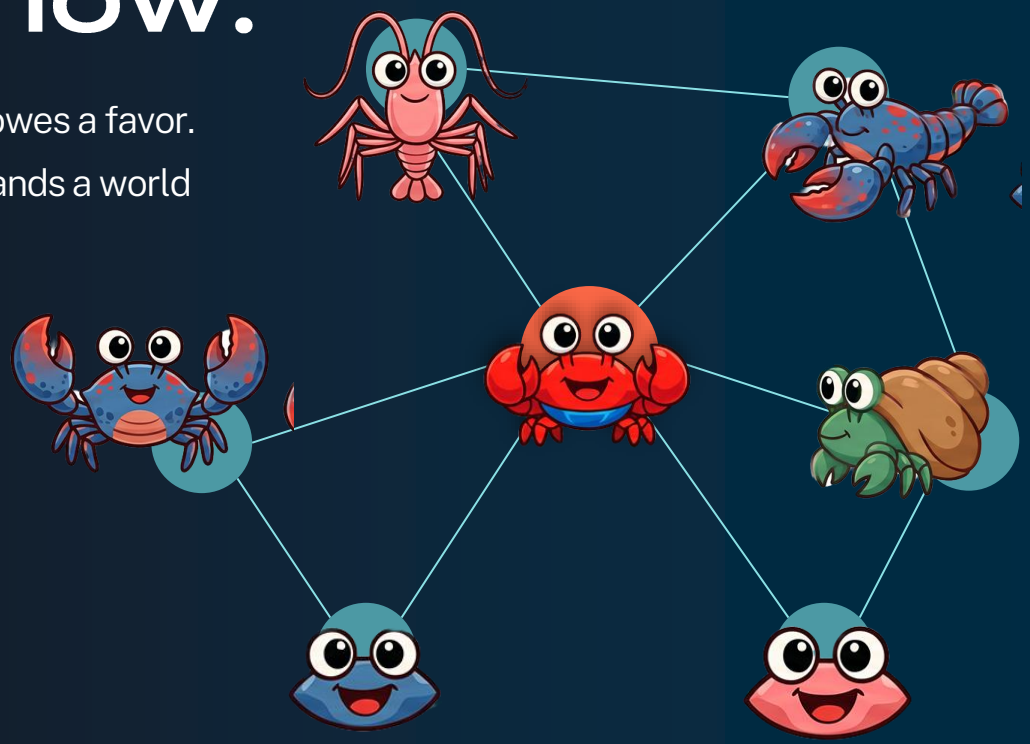
Picks the right skill — the graph says which one fits. Works the buffet end to end: crack → eat → next, the chain he never could before.

Multi-hop is free. Tool choice is grounded.



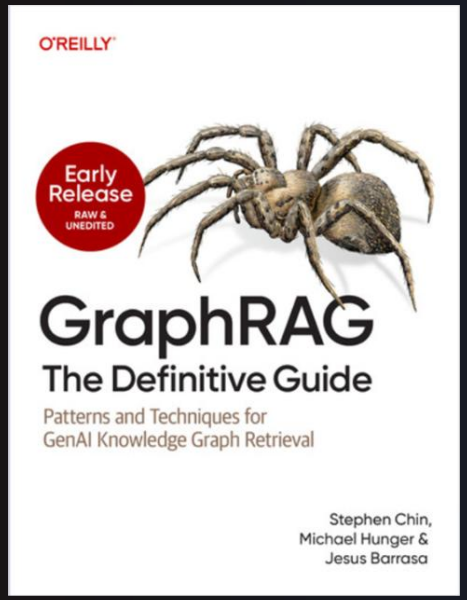
He remembers his whole crew now.

Who introduced whom. Who likes what. Who owes a favor. Not better recall — an assistant that understands a world made of relationships.



GraphRAG: The Definitive Guide

https://www.oreilly.com/library/view/graphrag-the-definitive/9798341630147/



By: Stephen Chin, Michael Hunger & Jesus Barrasa

December 2026

300 Pages

1h 3m

English

O'Reilly Media

neo4j Graph Intelligence Platform

Give your crab a graph.

Graph Academy

dev.neo4j.com/ga-rag

Stephen Chin
VP of Developer Relations
Neo4j · @steveonjava

neo4j Graph Intelligence Platform

Thank you

Stephen Chin (@steveonjava)
VP of Developer Relations @ Neo4j

@steveonjava